

Mauerstrassen Agent

Aktuelle Themen und Trends I und II
BWSE - SS 2026

Group 7

Dario-Benjamin Dzakic – 2310859034
Marco Magyar – 2410859018

Abstract—Tick the boxes for the level your implementation achieves:

- ☐ Minimal
- ☐ Advanced
- ☒ Expert

I. WORK DISTRIBUTION

Dario-Benjamin Dzakic designed and implemented the first three agents: *Strategy Advisor*, *Market Analyst*, and *Stock Screener*, together with their corresponding tasks (`strategy_task`, `trend_task`, `screening_task`). He built the sector-analysis RAG knowledge source (`sector_analysis.md`) and authored the Finnhub MCP server (`finnhub_server.py`), which exposes six financial-data tools (`get_company_profile`, `get_basic_financials`, `get_financials_reported`, `get_company_news`, `get_recommendation_trends`, `get_quote`).

Marco Magyar designed and implemented the remaining two agents: *Stock Checker* and *Portfolio Builder*, together with their corresponding tasks (`checker_task`, `portfolio_task`). He built the investment-strategy and risk-framework RAG knowledge sources (`investment_strategies.md`, `risk_frameworks.md`) and developed the MCP-CrewAI integration layer (`mcp_servers.py`) that connects the Finnhub MCP server to the CrewAI tool ecosystem via `MCPServerAdapter`. He also designed the overall crew orchestration, including the hierarchical process configuration with a manager LLM and the Google Generative AI embedding provider.

Both team members contributed to architecture design, testing, and documentation.

II. PROBLEM STATEMENT

Retail investors face a complex, information-rich landscape when constructing equity portfolios. They must assess their own risk tolerance, identify current market trends, screen relevant stocks, validate fundamentals, and finally allocate capital in a diversified manner. Each of these steps requires different expertise—from behavioural finance to fundamental analysis—and performing them manually is time-consuming and error-prone.

Mauerstrassen Agent is a multi-agent system built on CrewAI [?] that automates this end-to-end portfolio construction

workflow. Given a user's capital, risk tolerance, and investment horizon, the system produces a concrete, validated portfolio recommendation with position sizes, buy prices, and stop-loss levels.

Assumptions:

- The user has a brokerage account and can execute trades independently.
- Market data retrieved via the Finnhub API [?] and web searches (Serper) is sufficiently up-to-date for portfolio construction.
- The system produces *recommendations*, not financial advice in a regulatory sense.

III. ARCHITECTURE

The system is implemented as a CrewAI *crew* with five specialised agents orchestrated by a **hierarchical process**. A manager LLM coordinates agent execution, delegates tasks, and synthesises intermediate results. The architecture integrates three key technologies:

- 1) **RAG (Retrieval-Augmented Generation)**: Domain knowledge files are embedded and made available to agents at inference time via CrewAI's `TextFileKnowledgeSource`.
- 2) **MCP (Model Context Protocol)**: A custom Finnhub MCP server exposes real-time financial data tools over stdio transport, consumed by agents through `MCPServerAdapter`.
- 3) **External Tools**: The Serper web-search tool provides live market information for trend analysis and stock screening.

A. Agent Description

The system comprises five agents, each with a distinct role:

- 1) **Strategy Advisor** (`strategy_advisor`) — An investment strategy expert that interviews the user via `human_input` to collect capital, risk tolerance, and time horizon. It validates the user's profile against the RAG knowledge base (investment strategies, risk frameworks, sector analysis) to detect contradictions (e.g. aggressive risk with a 3-month horizon) and produces a structured investment strategy with recommended sectors.

Knowledge sources: `strategy_source` (investment strategies + risk frameworks), `sector_source` (sector analysis).

- 2) **Market Analyst** (`market_analyst`) — Searches the web using `SerperDevTool` for five current market trends aligned with the user’s strategy. Presents trends with rationales and lets the user select which to pursue via `human_input`.
Tools: `SerperDevTool`.
- 3) **Stock Screener** (`stock_screener`) — For each selected trend, searches the web for the top five stocks best positioned to benefit. Filters by market cap and liquidity appropriate to the risk profile.
Tools: `SerperDevTool`.
- 4) **Stock Checker** (`stock_checker`) — The validation analyst that uses **MCP tools** (Finnhub API) to pull company profiles, financial metrics, reported financials, news, and analyst recommendations for each candidate stock. Produces a go/no-go signal per stock.
Tools: All six Finnhub MCP tools + `SerperDevTool`.
- 5) **Portfolio Builder** (`portfolio_builder`) — Synthesises the validated stock list and the investment strategy into a final portfolio. Computes allocation percentages, share counts, buy prices, and stop-loss levels. Outputs a clean markdown table.
Knowledge sources: `strategy_source`.

B. Task Description

Five tasks form the pipeline, each assigned to one agent:

- 1) **Strategy Task** (`strategy_task`, agent: `strategy_advisor`) — Interviews the user and produces a structured investment strategy (capital, risk level, horizon, strategy type, 2–3 target sectors). Uses `human_input`.
- 2) **Trend Task** (`trend_task`, agent: `market_analyst`) — Searches for five market trends matching the strategy. User selects trends to pursue. Context: `strategy_task`. Uses `human_input`.
- 3) **Screening Task** (`screening_task`, agent: `stock_screener`) — Identifies up to five stocks per selected trend. Context: `strategy_task`, `trend_task`.
- 4) **Checker Task** (`checker_task`, agent: `stock_checker`) — Validates each stock candidate using Finnhub MCP tools and web search. Produces per-stock go/no-go reports. Context: `screening_task`.
- 5) **Portfolio Task** (`portfolio_task`, agent: `portfolio_builder`) — Builds the final portfolio as a markdown table with allocation, shares, prices, and stop-losses. Output is written to `output/portfolio_recommendation.md`. Context: `strategy_task`, `checker_task`.

C. Flow

The crew uses a **hierarchical process** (`Process.hierarchical`) where a manager LLM orchestrates the sequential execution of all five tasks. The interaction flow is:

- 1) The **manager** delegates the `strategy_task` to the Strategy Advisor, which interviews the user.
- 2) The strategy output feeds into the `trend_task`, delegated to the Market Analyst, which again involves the user.
- 3) Both previous outputs flow into the `screening_task`, delegated to the Stock Screener.
- 4) The screening results are passed to the `checker_task`, where the Stock Checker validates each candidate via MCP (Finnhub) and web search.
- 5) Finally, the manager delegates the `portfolio_task` to the Portfolio Builder, which receives the original strategy and the validation results to construct the final recommendation.

Task context dependencies ensure that each agent receives exactly the information it needs. The `human_input` flag on the first two tasks creates an interactive loop where the user actively shapes the investment direction.

IV. MINIMAL

The minimal functionality of the system comprises a working multi-agent crew with clearly separated roles:

- **Five specialised agents**, each with a distinct role, goal, and backstory defined in `agents.yaml`.
- **Five sequential tasks** defined in `tasks.yaml`, with explicit context dependencies ensuring information flows correctly through the pipeline.
- **Structured output:** The final portfolio task produces a Pydantic-validated `PortfolioRecommendation` model containing stocks, summary, total capital, and risk level.
- **CrewAI orchestration** via the `@CrewBase` decorator pattern, with agents and tasks wired together in `crew.py`.
- **External tool integration:** The `SerperDevTool` gives the Market Analyst and Stock Screener access to real-time web search results.

V. ADVANCED

Beyond the minimal system, the following advanced features are implemented:

- **Human-in-the-loop interaction:** The `strategy_task` and `trend_task` use `human_input=True`, allowing the user to provide investment parameters and select trends interactively. This ensures the system adapts to individual user needs rather than running fully autonomously.
- **Hierarchical process with manager LLM:** Instead of a simple sequential process, the crew uses `Process.hierarchical` with a dedicated manager model (`MANAGER_MODEL` environment variable). The manager coordinates task delegation, handles inter-agent communication, and synthesises results—mimicking a real investment team with a senior portfolio manager overseeing analysts.

- **Task context chaining:** Tasks declare explicit context dependencies (e.g. `portfolio_task` receives context from both `strategy_task` and `checker_task`), enabling rich information flow across the pipeline.
- **Markdown output artifact:** The `portfolio_task` writes its result to `output/portfolio_recommendation.md`, producing a persistent, human-readable deliverable.

VI. EXPERT

The expert-level features distinguish this system through deep integration of RAG and MCP:

A. RAG — Retrieval-Augmented Generation

Three curated knowledge documents are embedded and served to agents at inference time via CrewAI’s `TextFileKnowledgeSource`:

- `investment_strategies.md` — Covers value, growth, momentum, dividend, and index/passive strategies with key metrics and suitability profiles.
- `risk_frameworks.md` — Defines conservative, moderate, and aggressive risk levels with allocation limits, drawdown tolerances, and common profile contradictions to flag.
- `sector_analysis.md` — Analyses seven sectors (Technology, Healthcare, Energy, Financials, Consumer Discretionary, Consumer Staples, Industrials) with drivers, risks, and sector rotation patterns.

Two knowledge source objects are constructed: `strategy_source` (`investment_strategies` + `risk_frameworks`) and `sector_source` (`sector_analysis`). These are injected into the `strategy_advisor` (both sources) and the `portfolio_builder` (`strategy_source`), giving these agents grounded domain knowledge that goes beyond what the base LLM knows.

Embeddings are generated using Google Generative AI (`google-generativeai`) configured via environment variables, enabling semantic retrieval over the knowledge base.

B. MCP — Model Context Protocol

A custom **Finnhub MCP server** (`finnhub_server.py`) is implemented using the `FastMCP` framework. It exposes six financial-data tools over **stdio transport**:

- 1) `get_company_profile` — Company name, sector, industry, market cap, exchange.
- 2) `get_basic_financials` — P/E, P/B, ROE, margins, 52-week range, dividend yield.
- 3) `get_financials_reported` — As-reported income statement, balance sheet, cash flow (annual/quarterly).
- 4) `get_company_news` — Recent company news articles.
- 5) `get_recommendation_trends` — Analyst buy/hold/sell recommendation counts.

- 6) `get_quote` — Real-time stock quote (price, change, high/low, open, previous close).

The MCP server is integrated into CrewAI through `MCPServerAdapter` in `mcp_servers.py`, which spawns the server as a subprocess via `StdioServerParameters` and converts MCP tools into CrewAI-compatible tool objects. The `stock_checker` agent receives all six MCP tools alongside the web search tool, enabling it to pull real financial data for stock validation.

This architecture cleanly separates the data-access layer (MCP server) from the agent logic (CrewAI), following the Model Context Protocol standard for tool interoperability.

VII. REFLECTION

During development and testing we encountered several issues that highlight the practical challenges of building LLM-based multi-agent systems.

MCP context size of Finnhub responses. The Finnhub API returns large JSON payloads—especially `get_financials_reported`, which includes full income statements, balance sheets, and cash-flow data across multiple periods. When the `stock_checker` agent called this tool for several stocks in sequence, the accumulated context quickly grew very large. This caused the LLM to hit token limits or to lose track of earlier validation results, degrading the quality of the go/no-go assessments. A possible mitigation would be to add a summarisation step inside the MCP tool or to limit the number of reporting periods returned.

Human input requested but not declared on the crew. The `strategy_task` and `trend_task` set `human_input=True` at the task level, which prompts the user for input during execution. However, the crew itself was not explicitly configured to expect human interaction, which led to unexpected blocking behaviour in certain execution environments. Ensuring that the crew-level configuration and the task-level flags are aligned is an important lesson for future projects.

Stock count not consistently five. The `screening_task` instructs the Stock Screener to return the “top 5 stocks” per selected trend. In practice, the agent sometimes returned fewer or more than five—depending on how the LLM interpreted the prompt and the availability of suitable candidates in the search results. Strict enforcement of output cardinality is difficult with free-form LLM generation; a post-processing validation step or a stricter output schema could help enforce the constraint.

Structured output not always followed by Portfolio Builder. A Pydantic model (`PortfolioRecommendation`) was defined to structure the final output, but the Portfolio Builder did not consistently produce output that matched the schema. Fields were sometimes missing or formatted differently (e.g. allocation as a string instead of a number). This is a known limitation of relying on LLMs to produce structured output without explicit function-calling or JSON-mode enforcement—the

agent sometimes preferred to write free-form markdown instead of adhering to the schema.